

VinhoSend

Relatório do projeto e requisitos de segurança

Campo	Informação
Projeto	VinhoSend - Marketplace de vinhos
Objetivo	Permitir cadastro de vinhos, publicação de anúncios, encomenda por outro usuário e geração de nota fiscal simples.
Stack	PHP, MySQL, PDO, HTML, CSS, JavaScript simples e GitHub Actions.
Banco	SQL único em database/vinhosend_unico.sql.
Documento	Relatório final com requisitos implementados e pontos para demonstração.

Este relatório foi organizado para acompanhar a entrega do projeto e também servir como guia de apresentação. O foco é mostrar o que foi feito, onde cada requisito aparece no código e como o sistema pode ser testado durante a defesa.

Observação importante: a nota fiscal implementada no sistema é uma nota simples de controle interno da encomenda. Ela não substitui uma NF-e oficial emitida por órgão fiscal. Para o escopo acadêmico, ela registra valor, forma de pagamento, comprador, vendedor, vinho e endereço informado pelo comprador.

1. Visão geral do sistema

O VinhoSend é um marketplace simples de vinhos. O usuário pode criar uma conta, cadastrar vinhos com foto, publicar anúncios, receber solicitações de compra e acompanhar o status das vendas. Outro usuário pode entrar no marketplace, encomendar um vinho, informar a forma de pagamento e preencher o comentário com endereço de entrega.

Fluxo principal

1. Usuário vendedor cria uma conta e faz login.
2. Vendedor cadastra um vinho com nome, tipo, país, safra, nota, descrição e foto.
3. Vendedor publica o vinho no marketplace informando preço e quantidade.
4. Usuário comprador cria outra conta, acessa o marketplace e encomenda o vinho.
5. Comprador informa forma de pagamento e endereço no comentário da encomenda.
6. Sistema registra a compra, gera número de nota e reduz o estoque do anúncio.
7. Vendedor visualiza a solicitação recebida e marca o pedido como enviado.
8. Comprador acompanha em Minhas Compras, visualiza a nota fiscal e confirma o recebimento.

Páginas principais

Página	Função
index.html	Página inicial e apresentação do sistema.
registro.php	Cadastro de usuários.
login.php	Autenticação.
painel.php	Área Meus Vinhos, com cadastro, edição, foto e exclusão.
marketplace.php	Lista anúncios ativos e permite encomendar vinhos.
meus_anuncios.php	Área do vendedor: anúncios publicados, solicitações recebidas e envio.
minhas_compras.php	Área do comprador: pedidos feitos, status e nota fiscal.
nota_fiscal.php	Visualização/impressão da nota simples da encomenda.
links_uteis.php	Página com orientações básicas de compra, entrega e dúvidas.
contato.php	Página de contato do sistema.

2. Estrutura do projeto

A organização foi feita em camadas simples. Isso facilita a apresentação e evita que tudo fique misturado em um único arquivo.

Pasta/arquivo	Responsabilidade
config/	Conexão, sessão e funções de segurança.
controladores/	Recebe formulários e executa ações do sistema.
modelos/	Classes das entidades principais: Usuario, Vinho, AnuncioVinho e Compra.
repositorios/	Acesso ao banco de dados usando PDO e prepared statements.
servicos/	Validação, criptografia e upload de imagens.
visoes/	Partes reaproveitadas da interface, como cabeçalho e rodapé.
database/vinhosend_unico.sql	Criação do banco e tabelas.
tests/	Testes automatizados usados localmente e no GitHub Actions.
.github/workflows/tests.yml	Workflow que roda lint e testes no GitHub.

Arquitetura usada

O projeto segue uma estrutura equivalente a MVC. As telas ficam nos arquivos PHP principais; as ações de formulário ficam nos controladores; os dados ficam nos modelos; e as operações de banco ficam nos repositórios.

O padrão que pode ser demonstrado é Repository/DAO. Ele aparece na pasta repositorios/ e separa consultas SQL do restante do sistema. Também existe uma camada simples de serviços para validação, criptografia e upload.

3. Banco de dados

O projeto usa apenas um arquivo SQL para facilitar a instalação no XAMPP ou no ambiente do professor.

database/vinhosend_unico.sql

Tabela	Uso no sistema
usuarios	Cadastro, autenticação, tentativas de login e bloqueio temporário.
vinhos	Entidade principal cadastrada pelo usuário. Inclui imagem e descrição.
anuncios_vinhos	Anúncios publicados pelo vendedor no marketplace.
compras	Encomendas realizadas pelos compradores, com pagamento, endereço, nota e status.
auditoria	Registro de ações importantes, como login, logout, cadastro, compra e envio.

4. Requisitos de segurança

A tabela abaixo lista os 10 requisitos exigidos para o projeto. Cada item foi escrito para fazer sentido com o VinhoSend e para ser localizado no código durante a defesa.

R1 - STRIDE - Spoofing	
Requisito	Autenticar usuário com senha protegida e bloquear tentativas repetidas.
Por que foi adotado	Evita que alguém tente se passar por outro usuário por força bruta ou senha exposta.
Onde está no código	controladores/entrar.php; controladores/cadastrar_usuario.php; repositorios/UsuarioRepositorio.php; config/config.php
Como demonstrar	Tentar login errado várias vezes e mostrar o bloqueio. Depois mostrar password_hash/password_verify no código.

R2 - STRIDE - Tampering	
Requisito	Proteger ações de alteração com token CSRF.
Por que foi adotado	Evita que outro site force o navegador do usuário logado a cadastrar, excluir ou comprar sem intenção.
Onde está no código	config/seguranca.php; controladores/salvar_vinho.php; controladores/comprar_anuncio.php; controladores/sair.php
Como demonstrar	Mostrar o campo csrf_token nos formulários e a validação nos controladores.

R3 - STRIDE - Elevation of Privilege	
Requisito	Impedir acesso a recursos de outro usuário.
Por que foi adotado	Evita que um usuário edite vinho, anúncio, compra ou nota que não pertence a ele.
Onde está no código	config/sessao.php; repositorios/VinhoRepositorio.php; repositorios/AnuncioRepositorio.php; repositorios/CompraRepositorio.php
Como demonstrar	Entrar com dois usuários e mostrar que cada um vê apenas seus próprios dados.

R4 - Codificação segura - CWE-89	
Requisito	Usar prepared statements em consultas SQL.
Por que foi adotado	Reduz risco de SQL Injection em cadastro, login, filtros, compras e anúncios.
Onde está no código	repositorios/UsuarioRepositorio.php; repositorios/VinhoRepositorio.php; repositorios/AnuncioRepositorio.php; repositorios/CompraRepositorio.php
Como demonstrar	Abrir os repositórios e mostrar prepare/execute em vez de concatenar SQL com dados do usuário.

R5 - Codificação segura - CWE-79	
Requisito	Escapar dados exibidos no HTML.
Por que foi adotado	Reduz risco de XSS quando nomes, comentários, descrições ou títulos aparecem na tela.
Onde está no código	config/seguranca.php; painel.php; marketplace.php; meus_anuncios.php; minhas_compras.php; nota_fiscal.php
Como demonstrar	Mostrar a função escapar() e um campo exibido com essa função.

R6 - Codificação segura - OWASP File Upload	
Requisito	Validar upload de foto do vinho.
Por que foi adotado	Evita envio de arquivo perigoso ou fora do tipo esperado.
Onde está no código	servicos/UploadImagemServico.php; uploads/.htaccess; uploads/vinhos/.htaccess; controladores/salvar_vinho.php
Como demonstrar	Tentar enviar arquivo não permitido e mostrar validação de tipo, tamanho e nome seguro.

R7 - Livre - Login obrigatório	
Requisito	Bloquear páginas internas para usuário não autenticado.
Por que foi adotado	Impede uso do marketplace, compras e área do vendedor sem sessão válida.
Onde está no código	config/sessao.php; painel.php; marketplace.php; meus_anuncios.php; minhas_compras.php; nota_fiscal.php
Como demonstrar	Acessar uma página interna sem login e mostrar o redirecionamento para login.php.

R8 - Livre - Sessão segura	
Requisito	Configurar cookie de sessão com opções de segurança.
Por que foi adotado	Reduz exposição de sessão no navegador e dificulta abuso básico de cookie.
Onde está no código	config/sessao.php
Como demonstrar	Mostrar session_set_cookie_params com httponly e samesite.

R9 - Livre - Criptografia	
Requisito	Cifrar textos sensíveis antes de salvar no banco.
Por que foi adotado	Protege descrições, observações e endereço de entrega armazenados no banco.
Onde está no código	servicos/CriptografiaServico.php; repositorios/VinhoRepositorio.php; repositorios/AnuncioRepositorio.php; repositorios/CompraRepositorio.php
Como demonstrar	Mostrar AES-256-GCM, openssl_encrypt/openssl_decrypt e o teste de criptografia.

R10 - Livre - Auditoria	
Requisito	Registrar ações importantes do sistema.
Por que foi adotado	Ajuda a rastrear login, logout, cadastro, anúncio, compra, envio e conclusão.
Onde está no código	repositorios/AuditoriaRepositorio.php; controladores/*.php; database/vinhosend_unico.sql
Como demonstrar	Mostrar tabela auditoria e chamadas registrar() nos controladores.

Referências usadas nos requisitos

As referências foram escolhidas por serem comuns em segurança de aplicações web e por combinarem com os controles implementados no projeto: OWASP ASVS, OWASP Cheat Sheet Series, MITRE CWE-89, MITRE CWE-79, OWASP CSRF Prevention Cheat Sheet, OWASP File Upload Cheat Sheet e OWASP Session Management Cheat Sheet.

5. Criptografia

O módulo de criptografia foi implementado para não deixar o projeto depender apenas de hash de senha. O serviço fica separado em `servicos/CriptografiaServico.php` e usa AES-256-GCM com OpenSSL.

Ponto	Descrição
Algoritmo	AES-256-GCM.
Funções usadas	<code>openssl_encrypt</code> e <code>openssl_decrypt</code> .
Dados protegidos	Descrições, observações e endereço de entrega da encomenda.
Formato salvo	Texto cifrado com prefixo de versão, IV, tag e conteúdo cifrado.
Teste	<code>tests/CriptografiaServicoTest.php</code> confirma que o texto cifrado fica diferente do original e volta corretamente ao decifrar.

Durante a defesa, a explicação pode ser direta: antes de salvar um texto sensível, o repositório chama o serviço de criptografia; para exibir ao usuário autorizado, o repositório decifra o valor. A senha do usuário não é decifrada, pois senha usa hash, que é outro tipo de proteção.

6. Testes e GitHub Actions

Os testes ficam na pasta `tests/` e são executados com `php tests/run.php`. Eles não testam só se o arquivo existe; também verificam regras de segurança e de negócio.

Teste	O que cobre
<code>AutenticacaoTest.php</code>	Hash e verificação de senha.
<code>CriptografiaServicoTest.php</code>	Criptografia e decifragem com retorno do conteúdo original.
<code>LoginObrigatorioTest.php</code>	Presença de exigência de login nas páginas internas.
<code>MarketplaceTest.php</code>	Regras de preço, quantidade, compra e status.
<code>RepositorySecurityTest.php</code>	Prepared statements, filtros por usuário e uso de criptografia.
<code>UploadImagemServicoTest.php</code>	Validação de imagem, extensões, <code>.htaccess</code> , SQL único, nota e status.
<code>ValidadorTest.php</code>	Validação de e-mail, senha, safra, nota e campos obrigatórios.

```
php tests/run.php
```

Resultado esperado: todos os testes devem aparecer com [OK] e a mensagem final deve indicar 7 testes executados com sucesso.

Workflow no GitHub

O arquivo `.github/workflows/tests.yml` roda automaticamente no push e no pull request. Ele confere o SQL único, faz lint dos arquivos PHP e executa os testes.

```
.github/workflows/tests.yml
```

7. Checklist de entrega

OK	Item a conferir	Como validar
	Repositório e arquivos	
[OK]	Repositório público abre normalmente.	Abrir a página do GitHub antes da apresentação.
[OK]	ZIP final contém o código completo.	Conferir se estão presentes config, controladores, modelos, repositórios, serviços, tests, database e docs.
[OK]	Pasta docs contém o PDF.	Conferir docs/relatorio-requisitos-seguranca.pdf.
	Banco e execução	
[OK]	SQL único importa no phpMyAdmin.	Importar database/vinhosend_unico.sql em banco limpo.
[OK]	Cadastro de usuário funciona.	Criar um vendedor e um comprador.
[OK]	Login funciona e páginas internas exigem sessão.	Acessar marketplace.php sem login e confirmar redirecionamento.
[OK]	Upload de foto do vinho funciona.	Cadastrar vinho com JPG, PNG ou WEBP válido.
	Marketplace	
[OK]	Vendedor publica anúncio.	Cadastrar vinho e publicar em Vender.
[OK]	Comprador encomenda vinho.	Entrar com segundo usuário e comprar no Marketplace.
[OK]	Nota fiscal simples aparece.	Abrir nota_fiscal.php a partir de Minhas Compras.
[OK]	Vendedor marca como enviado.	Entrar com vendedor e atualizar status do pedido.
[OK]	Comprador confirma recebimento.	Entrar com comprador e concluir compra.
	Segurança e testes	
[OK]	PDF lista os 10 requisitos.	Abrir o relatório na seção 4.
[OK]	Requisitos aparecem no código.	Usar a coluna Onde está da tabela de requisitos.
[OK]	Testes passam localmente.	Rodar php tests/run.php.
[OK]	GitHub Actions está verde.	Abrir aba Actions e conferir a última execução.

8. Roteiro para a defesa

Este roteiro é curto para evitar se perder durante a apresentação. Cada etapa mostra uma parte da rubrica.

1. Abrir o GitHub e mostrar a estrutura do projeto.
2. Mostrar o PDF em docs/relatorio-requisitos-seguranca.pdf.
3. Mostrar o SQL único em database/vinhosend_unico.sql.
4. Rodar ou mostrar o resultado de php tests/run.php.
5. Abrir a aba Actions e mostrar o workflow verde.
6. Criar ou usar usuário vendedor e cadastrar um vinho com foto.
7. Publicar esse vinho em Vender / Meus Anúncios.
8. Entrar com usuário comprador e encomendar o vinho no marketplace.
9. Informar forma de pagamento e endereço no comentário.
10. Abrir a nota fiscal simples.
11. Voltar no vendedor e marcar a compra como enviada.
12. Voltar no comprador e confirmar recebimento.
13. Abrir o código e mostrar um requisito de cada grupo: STRIDE, codificação segura e livre.

9. Pontos que devem ser explicados com cuidado

Alguns pontos podem gerar pergunta. É melhor explicar com clareza em vez de tentar vender como algo maior do que é.

Ponto	Explicação correta
Nota fiscal	É uma nota simples de controle da venda no sistema. Não é integração com SEFAZ e não é NF-e oficial.
Pagamento	O sistema registra a forma de pagamento escolhida, mas não processa pagamento real por gateway.
Upload de imagem	Há validação de extensão, MIME, tamanho, nome seguro e bloqueio de PHP via .htaccess. Não há antivírus no upload.
Criptografia	É usada para proteger textos sensíveis no banco. Senha usa hash, não criptografia reversível.
Banco	O SQL único recria a base. Para apresentação, isso facilita começar com ambiente limpo.
Git	O histórico, branches e Actions devem ser mostrados no GitHub até a data de entrega.

10. Perguntas prováveis e respostas curtas

Pergunta	Resposta sugerida
Qual é a entidade principal?	A entidade principal é Vinho. A partir dela são criados anúncios e compras.
Onde está o MVC?	Modelos em modelos/, controladores em controladores/, telas nos PHP principais/visoes e persistência em repositórios/.
Qual pattern foi usado?	Repository/DAO para separar acesso ao banco de dados.
Como o sistema evita SQL Injection?	Usando PDO com prepare e execute nos repositórios.
Como o sistema evita acesso indevido?	As páginas internas exigem login e os repositórios filtram por usuario_id, comprador_id ou vendedor_id.
Onde está a criptografia?	Em serviços/CriptografiaServico.php, usando AES-256-GCM.
O que os testes validam?	Autenticação, criptografia, login obrigatório, marketplace, repositórios, upload e validações.

11. Conferência final

Se todos os itens abaixo estiverem marcados, o projeto está pronto para entregar e apresentar.

- [OK] PDF dentro de docs/relatorio-requisitos-seguranca.pdf.
- [OK] README.md atualizado na raiz do projeto.
- [OK] database/vinhosend_unico.sql presente e importável.
- [OK] tests/run.php executa com sucesso.
- [OK] GitHub Actions com última execução verde.
- [OK] Cadastro, login, vinho, anúncio, compra, envio, conclusão e nota fiscal testados manualmente.

12. Referências

- OWASP Application Security Verification Standard (ASVS) - requisitos técnicos de segurança para aplicações web.
- OWASP Cheat Sheet Series - Password Storage, Session Management, CSRF Prevention, XSS Prevention e File Upload.
- MITRE CWE-89 - Improper Neutralization of Special Elements used in an SQL Command.
- MITRE CWE-79 - Improper Neutralization of Input During Web Page Generation.
- Documentação do PHP - password_hash, password_verify, PDO, openssl_encrypt e openssl_decrypt.